

Article

Impact of Developer Queries on the Effectiveness of Conversational Large Language Models in Programming

Viktor Taneski , Sašo Karakatič , Patrik Rek  and Gregor Jošt

Faculty of Electrical Engineering and Computer Science, University of Maribor, Koroška Cesta 46, 2000 Maribor, Slovenia; saso.karakatic@um.si (S.K.); patrik.rek@um.si (P.R.); gregor.jost@um.si (G.J.)

* Correspondence: viktor.taneski@um.si

Abstract: This study investigates the effects of LLM-based coding assistance on web application development by students using a frontend framework. Rather than comparing different models, it focuses on how students interact with LLM tools to isolate the impact of query type on coding success. To this end, participants were instructed to rely exclusively on LLMs for writing code, based on a given set of specifications, and their queries were categorized into seven types: Error Fixing (EF), Feature Implementation (FI), Code Optimization (CO), Code Understanding (CU), Best Practices (BP), Documentation (DOC), and Concept Clarification (CC). The results reveal that students who queried LLMs for error fixing (EF) were statistically more likely to have runnable code, regardless of prior knowledge. Additionally, students seeking code understanding (CU) and error fixing performed better, even when normalizing for previous coding ability. These findings suggest that the nature of the queries made to LLMs influences the success of programming tasks and provides insights into how AI tools can assist learning in software development.

Keywords: large language models (LLMs); prompt engineering; query type analysis; AI-assisted programming; educational software development



Academic Editors: Francisco García-Sánchez and Minoru Sasaki

Received: 5 May 2025

Revised: 11 June 2025

Accepted: 14 June 2025

Published: 17 June 2025

Citation: Taneski, V.; Karakatič, S.; Rek, P.; Jošt, G. Impact of Developer Queries on the Effectiveness of Conversational Large Language Models in Programming. *Appl. Sci.* **2025**, *15*, 6836. <https://doi.org/10.3390/app15126836>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

The rapid advancement of artificial intelligence (AI) has brought transformative changes across multiple domains, particularly in software development and education. Among these innovations, large language models (LLMs) have demonstrated remarkable capabilities in natural language understanding, code generation, and context-specific problem solving [1,2]. These generative AI systems hold significant promise in supporting software development tasks by offering real-time feedback, debugging assistance, and concept clarification. However, the practical effectiveness of LLM-based tools in fostering learning and aiding students during software development tasks remains an area of active investigation.

In the domain of software engineering education, integrating LLMs has been shown to improve productivity in specific tasks, such as foundational code generation and debugging, particularly in the early stages of development [2]. Furthermore, these tools have been found to play a complementary role in higher education by aligning with the pedagogical principles of Education 4.0 [3], enabling autonomous, collaborative, and interactive learning environments while highlighting the need for human oversight to maintain quality and accuracy. Despite their potential, limited research examines how students interact with LLMs during coding tasks and how their query formulation and prior experience influence learning outcomes and task success.

Similar concerns have been raised in broader software engineering contexts, where studies using datasets like DevGPT reveal important insights about the types of tasks developers delegate to ChatGPT, how effectively those tasks are handled, and how prompt quality influences interaction success [4].

This study seeks to address this gap by investigating the effects of LLM-based coding assistance on web application development by students using a frontend framework. Specifically, we analyze how the type of queries made to LLMs impacts the effectiveness of LLM assistance. These queries are categorized into seven types: Error Fixing (EF), Feature Implementation (FI), Code Optimization (CO), Code Understanding (CU), Best Practices (BP), Documentation (DOC), and Concept Clarification (CC).

Prior work has shown that interactions with LLMs influence the outcomes of software development tasks, suggesting a need for more effective interaction mechanisms [1]. Building on these findings, we explore whether prior coding experience modifies the benefits of LLM assistance and investigate the extent to which query types predict successful task completion. The research questions guiding this study are as follows:

- RQ1: How does using LLM-based coding assistance impact the likelihood of students producing runnable code, considering their prior coding experience?
- H1: Using LLM-based coding assistance increases the likelihood of students producing runnable code, regardless of prior coding experience.
- RQ2: How does the formulation of queries when using LLM-based coding assistance influence the effectiveness of the assistance?
- H2: Query formulation significantly influences the effectiveness of LLM-based coding assistance.
- RQ3: How does prior coding experience affect the benefits students receive from LLM-based coding assistance?
- H3: Students with less prior coding experience benefit more from LLM-based coding assistance than experienced students.

By examining these questions, this study provides empirical insights into the role of LLM-based tools in software engineering education, offering evidence on how query formulation and prior experience shape their impact. The study introduces novel methodological approaches to assess the impact of LLMs on software development education. A key innovation is the dual-phase design, which compares performance with and without LLM assistance within the same group of participants. This within-subjects framework allows for a clearer, more controlled analysis of LLM's impact on student outcomes. Additionally, the study leverages query tagging and component hierarchy diagrams, providing new tools to evaluate the quality of student prompts and the complexity of their coding tasks. These methodologies offer a more granular understanding of how specific query types influence learning outcomes and task success, thus advancing the field of AI-assisted coding education. By combining these techniques, this research offers unique insights into the practical application of LLMs in real-world educational settings.

2. Research Background

The increasing adoption of LLMs in education raises questions about their role in learning processes, particularly in technical domains like computer science. Tools like LLMs offer several benefits, including instant code generation, problem-solving advice, and explanations of abstract programming concepts. However, evaluating whether such tools enhance learning or inadvertently enable dependency on AI for problem solving without proper understanding is essential.

In the context of software engineering, the use of structured prompting across different phases of the Software Development Life Cycle (SDLC) has shown that Generative AI

techniques like ChatGPT can reduce skill barriers and accelerate development by improving the quality of responses, depending on the quality of the initial prompts [5]. Moreover, several design techniques, particularly prompt patterns, are being explored to automate common software engineering tasks [6]. These patterns address issues like ensuring code decoupling from third-party libraries, creating API specifications from requirements lists, and improving requirements elicitation, rapid prototyping, code quality, deployment, and testing.

LLMs have rapidly transformed software development, especially in code generation. However, their inconsistent performance, propensity to hallucinations, and quality issues complicate program comprehension and hinder maintainability. Research [7] indicates that prompt engineering, the practice of designing inputs to direct LLMs towards generating relevant outputs, may help address these challenges. In this regard, researchers have introduced prompt patterns and structured templates to guide users in formulating their requests. However, the influence of prompt patterns on code quality has yet to be thoroughly investigated. An improved understanding of this relationship would be essential to advancing our collective knowledge on how to effectively use LLMs for code generation, thereby enhancing their understandability in contemporary software development. This paper empirically investigates the impact of prompt patterns on code quality, specifically maintainability, security, and reliability, using the Dev-GPT dataset. Results show that Zero-Shot prompting is most common, followed by Zero-Shot with Chain-of-Thought and Few-Shot. Analysis of 7583 code files across quality metrics revealed minimal issues. Kruskal–Wallis tests indicated no significant differences among patterns, suggesting that prompt structure may not substantially impact these quality metrics in ChatGPT-assisted code generation.

Recent work in prompt engineering has identified several standard interaction modes for eliciting effective responses from LLMs. Notably, Brown et al. [8] introduced zero-shot and few-shot prompting strategies, where task instructions are given with or without examples, respectively. Later studies extended this taxonomy to include methods such as instruction-based prompting [9] and chain-of-thought prompting for eliciting reasoning steps [10]. These frameworks provide useful guidance for understanding how prompt design impacts LLM output quality and reliability. However, most prior work has focused on performance in benchmark tasks (e.g., reasoning, translation, math), while relatively few studies have investigated how novice users, such as students, naturally formulate prompts during open-ended software development tasks.

Building on the growing interest in prompt engineering, recent research has proposed the use of prompt pattern catalogues as structured, reusable solutions to common interaction challenges with LLMs. Similar to software design patterns, these prompt patterns help guide users towards more effective and efficient prompting strategies across various domains, including software development and education. This approach moves beyond individual prompt examples to classify and document broader interaction strategies, enabling users to develop mental models for structuring prompts [11]. For instance, combining prompt types such as Game Play and Visualization Generator has enabled complex LLM outputs like cybersecurity simulations. However, researchers also stress that the utility of such catalogues will evolve alongside LLM capabilities and that further refinement and domain-specific pattern development is necessary to meet emerging needs.

While structural approaches to prompt engineering (e.g., pattern catalogues) help users formulate more effective queries, recent research [12] emphasizes the interpretive aspect of prompting by applying a hermeneutic lens to LLM interactions. This work explores how prompt complexity and specificity shape not just output accuracy but also the perceived meaningfulness or hermeneuticity of AI-generated texts. Findings suggest that although

clearer, more directive prompts may improve technical precision, they often reduce the depth of interpretive engagement, limiting novel or unexpected insights. Conversely, vague prompts sometimes yield more creative, albeit less focused, responses. These dynamics highlight the importance of balancing user intent, prompt design, and reader interpretation, particularly in educational contexts where meaning making is central to learning.

In parallel, LLMs are increasingly influencing the field of software engineering, as noted by Liu et al. [13]. By creating LLM-based agents that go beyond the capabilities of standalone models, LLMs are now assisting software developers by integrating external tools and resources to solve real-world software engineering problems. This marks a shift towards multi-agent systems and more robust human–agent interactions, which offer promising solutions to the complex nature of software development. However, these advancements also highlight challenges related to integrating LLMs with human expertise, suggesting that a balanced synergy between AI and human oversight is crucial to fully harness the potential of LLM-based agents.

The integration of LLMs into higher education can be further understood through the lens of constructivist learning theory, which posits that knowledge is actively constructed by learners through experience, reflection, and social interaction. This theoretical foundation aligns closely with the core principles of Education 4.0, namely autonomy, collaboration, and innovation in learning. LLMs support autonomous learning by enabling learners to explore content independently, ask questions in natural language, and receive immediate, context-sensitive feedback, fostering metacognition and self-directed learning in line with heutagogical principles. Simultaneously, LLMs facilitate collaborative learning by supporting peer discussion, the co-creation of knowledge, and group-based inquiry, as encouraged by peeragogy. Moreover, by mediating interaction within digital environments, LLMs contribute to cyberpedagogical practices that emphasize interactivity and the integration of digital tools into the learning process. In this way, the application of LLMs in educational settings not only exemplifies the pedagogical shifts advocated by Education 4.0 but is also deeply rooted in constructivist theory, providing a robust theoretical basis for their role in fostering meaningful, student-centred learning experiences [3].

Together, these studies underline the transformative potential of LLMs, from supporting startup innovation to revolutionizing software engineering practices. However, they also highlight the ongoing challenges of ensuring that AI-generated outputs are both reliable and effectively integrated into human workflows. Whether in startups or software development, the human element remains indispensable, not only in providing oversight and critical thinking but also in shaping how AI tools are applied in practice. This notion aligns with our own work, where we asked students to implement a functioning app using only LLMs without directly writing any code. This experiment demonstrated both the potential and the limitations of LLMs as standalone tools, reinforcing the idea that while they can automate complex tasks and reduce the need for traditional coding, human judgment and careful prompt design remain crucial for achieving effective outcomes.

While prior research has provided valuable insights into the capabilities of LLMs for code generation, debugging, and requirements elicitation, most studies focus on the functional output of these tools or explore prompt engineering in general-purpose or synthetic contexts (e.g., Dev-GPT datasets). However, a critical gap remains in understanding how the formulation of developer queries affects learning effectiveness and task success, especially among novice programmers engaged in real-world development using modern frontend frameworks.

Our work addresses this gap by offering an empirical, query-type-level analysis of LLM usage within a controlled educational setting. Unlike prior studies, we focus not only on LLM performance but also on the interaction dynamics between students and

the AI system. We categorize student queries into seven types and statistically evaluate how each relates to runnable code and assignment success while controlling for prior experience. This fine-grained approach reveals nuanced patterns that are often overlooked in broader classifications.

By analyzing authentic, student-generated queries embedded in actual coursework, we demonstrate both the potential and the limitations of LLMs as educational tools. While LLMs can automate complex programming tasks and aid conceptual understanding, human judgment and deliberate prompt design remain essential. This pedagogical framing contributes not only to best practices in AI-assisted curriculum design but also to the broader discourse on cultivating effective prompting skills as a facet of digital literacy. Ultimately, our study underscores the importance of integrating AI thoughtfully into human workflows, where technology complements critical thinking and software engineering expertise.

3. Materials and Methods

The experiment involved thirty-seven second-year undergraduate students enrolled in a web development course who were tasked with creating a web application using a specified frontend framework. Second-year undergraduate students were carefully considered as participants for this study to align with the research objectives. They were chosen due to their foundational knowledge in web development, including HTML5, CSS3, and vanilla JavaScript ES6, which they had gained through their first-year coursework. Although they had a basic understanding, they had not been formally introduced to React (Meta Platforms, Inc., Menlo Park, CA, USA), a widely used JavaScript library for developing dynamic user interfaces. This made them ideal subjects for assessing the effectiveness of using LLMs to aid in learning new technology. The ten-week experiment was structured into specific phases, as outlined in Figure 1.

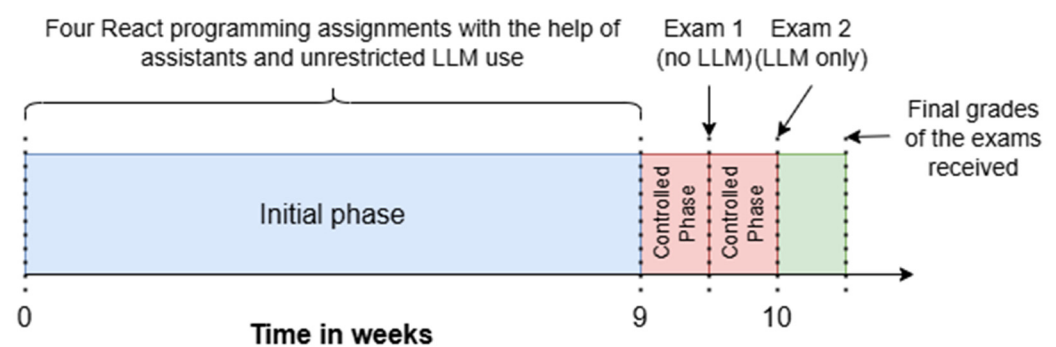


Figure 1. Overview of the experiment phases. The diagram illustrates the sequence of activities during the study, including the initial assignment phase and the two-part controlled phase. It highlights the transition from informal LLM use to controlled conditions where LLMs were either prohibited or required, allowing for a comparative analysis of their impact on student performance.

The time allocation in Figure 1 was based on the academic calendar structure of the university. The 9-week initial phase corresponded to weekly instructional modules in a semester-long course, which aligned with the university's semester structure and followed established educational practices [14], allowing students to progressively build programming proficiency. The final week was reserved for the controlled testing phase, in which students completed two time-constrained exams under systematically varied LLM usage conditions. This design ensured both ecological validity and controlled comparison.

While the procedures used in this study align with standard experimental practices, the novelty lies in their strategic application to LLM-assisted coding education. The dual-phase design, contrasting LLM-only and no-LLM performance within the same

participants, provides a clean, within-subjects framework to isolate the impact of LLM assistance. Additionally, the use of query tagging and component hierarchy diagrams introduces innovative means of analyzing prompt quality and assignment complexity in LLM contexts. Together, these elements allow for a nuanced and controlled analysis of LLM effectiveness in a realistic yet evaluative educational setting.

Given the increasing prevalence of LLMs in the programming domain, we allowed students to employ any LLM tool they deemed suitable for completing their programming assignments in the initial phase. We adopted this approach to mimic authentic programming environments where developers frequently leverage such tools to enhance their productivity and problem-solving capabilities.

To assess the influence of LLM utilization on students' learning experiences, we implemented a controlled phase consisting of two parts. The first part was an exam, where using LLMs was prohibited. This phase was designed to isolate the effects of LLMs on learning outcomes by eliminating their presence during specific learning tasks. By comparing students' performance and perceptions across the unrestricted and controlled phases, we aimed to observe the extent to which LLMs contribute to educational effectiveness.

The second part was an exam, which took place during the same week as the first exam. This time, the students were instructed to rely solely on LLMs to generate and write their code and investigate the effects of LLM-based coding assistance on web application development. By comparing the assignment grade from Exam 1 to the grade from Exam 2, we eliminate the impact of prior knowledge, allowing us to focus solely on the difference contributed by an LLM. Our goal was not to measure students' overall performance based on prior knowledge but rather to isolate the effect of LLM assistance on their output. To ensure that differences in individual programming ability did not affect the results, we analyzed the correlation between prior assignment scores and exam performance. The correlation was weak and not statistically significant (Pearson $r = -0.19$, $p = 0.26$), suggesting that baseline skill level did not substantially influence outcomes. This supports the validity of our within-subjects design in capturing the impact of LLM use.

While previous studies have explored various aspects of prompt engineering and user interactions with LLMs, including understanding end-user intuitions and behaviours in prompt design [15], analyzing students' prompting strategies and their effects on model response accuracy [16], and implementing AI literacy interventions to improve users' understanding and effective use of LLMs [17], these works primarily focus on descriptive analyses of prompting behaviour or educational interventions at a high level. In contrast, our study introduces a novel analytical framework that systematically combines query tagging and component hierarchy diagrams to quantitatively evaluate both the quality of student prompts and the structural complexity of coding tasks in the context of LLM-assisted software engineering education. This approach allows for a granular, data-driven analysis of how specific prompt types interact with task complexity to influence learning outcomes, an aspect that, to the best of our knowledge, has not been previously investigated. By applying this integrated framework within a controlled experimental design contrasting LLM-assisted and non-LLM-assisted conditions, our work advances the understanding of LLMs' practical impact on student coding performance and learning processes.

3.1. Initial Phase

During the initial nine-week phase, students completed four assignments which focused on developing React applications using TypeScript (Microsoft, Redmond, WA, USA). These assignments explored different aspects of React, including component-based architecture, state management, and routing. A key requirement was for students to leverage TypeScript's typing system to improve code quality and maintainability.

Each assignment introduced progressively more complex challenges, building on concepts from the previous tasks. The first assignment involved creating a basic React component hierarchy, while the later ones addressed more advanced topics like routing, state management, and lifting state up. Real-world scenarios were integrated into the assignments to simulate industry-relevant challenges, encouraging students to develop problem-solving skills within the context of modern web development.

Throughout these tasks, participants were permitted to use LLMs informally for various purposes, such as debugging, seeking additional explanations, or performing any task they found helpful. They also received guidance from two experienced assistants, each with over 10 years of programming expertise, who provided support as needed. This informal use of LLMs, combined with assistant support, reflected real-world scenarios where students might use AI tools alongside human assistance in their learning process.

3.2. Controlled Phase

After completing the four assignments over nine weeks, the study entered its controlled and experimental phase in week ten. In this phase, we aimed to examine how ChatGPT can help close the performance gap on more challenging coding problems, regardless of the developer's existing knowledge or abilities. Therefore, this phase was divided into two parts.

The first part was an exam where participants were given a carefully structured assignment where using LLMs was strictly prohibited. However, considering the nature of the course (an introduction to developing web applications using React), participants were permitted to use Google and official React documentation as supplementary resources. Within a one-hour window, they were required to implement an application using React. The assignment was intentionally designed so that participants already had the knowledge and skills necessary to complete it. It reflected concepts and challenges they had previously encountered and mastered during the study, without introducing any new material. The goal was to create a fair and controlled environment to evaluate how participants performed a familiar task independently, without the assistance of LLMs, within a set time limit.

Within the same week as the first exam, the second part of the controlled phase took place. The second part was another exam, where an LLM was permitted. Participants were restricted from writing any code themselves and were instructed to rely solely on LLM to generate and refine their code. A set of specifications was provided, defining the features and functionality of the application. Appendix A provides the instructions for the assignments used in the controlled phase.

Throughout both assignments, two experienced assistants closely supervised the participants to ensure compliance with the no-LLM rule and later with the LLM-only rule, maintaining the experiment's integrity.

The experiment was graded based on two criteria: functionality and correctness. Functionality (the first part of the grade) was a straightforward "Yes" or "No", indicating whether the solution ran without any errors. Correctness (the second part of the grade) was more detailed, with multiple subcriteria determining the score. A full breakdown of these subcriteria can be found in Appendix B.

3.3. Complexity of the Assignments

To provide context for the results in the following section, we offer an overview of the complexity of the assignments completed by participants during the experiment. For this purpose, we used a Component Hierarchy Diagram, which, we argue, is the best choice because it visually aligns with React's component-based architecture. It emphasizes

modularity and the relationships between parts of the app, directly reflecting the complexity of implementation.

For optimization purposes, we will include a component hierarchy diagram solely for the final assignment from the initial phase. This assignment comprehensively encapsulates all the requirements from previous tasks, integrates additional elements, and requires applying knowledge acquired through the completion of the preceding three assignments. Figure 2a represents the component hierarchy diagram for the fourth assignment of the initial phase.

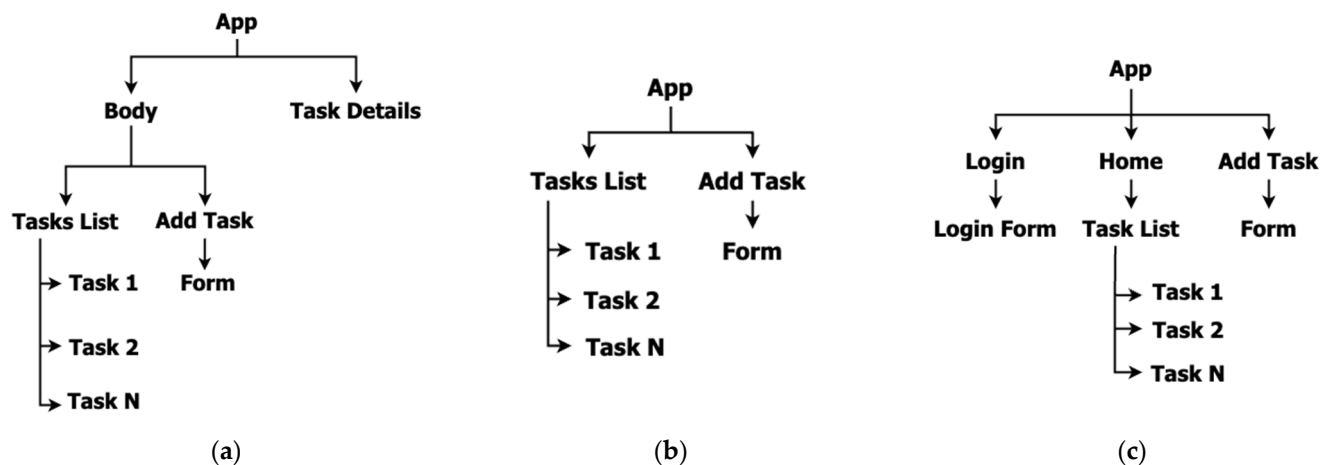


Figure 2. Component hierarchy diagrams for selected assignments. (a) Final assignment from the initial phase, combining all prior requirements and showcasing full React component complexity. (b) Exam 1 task (LLM-prohibited), slightly simplified to reflect time constraints but still covering essential React concepts. (c) Exam 2 task (LLM-mandated), incorporating more features due to LLM assistance but with a different complexity profile.

The assignment in Exam 1 of the controlled phase was slightly more straightforward than the last one in the initial phase. This was because we were limited by time (the exam was performed within one hour) and because LLMs during the exam were not allowed, which added to the complexity of the assignment. It is important to note that, regardless, the assignment incorporated all the necessary concepts and challenges one must master to work with React properly. Figure 2b shows the component hierarchy diagram for the assignment from Exam 1 of the controlled phase.

Finally, the last assignment from Exam 2 of the controlled phase was expectedly slightly more complex than the first two since it encompassed only the usage of LLMs, which simplified the assignment. The diagram in Figure 2c shows the component organization of the last assignment from Exam 2 of the controlled phase of the experiment.

To quantitatively assess the complexity of the programming tasks, we measured the cyclomatic complexity of representative student solutions using SonarQube Cloud (SonarSource, Geneva, Switzerland) [18]. The tasks from both the Initial Phase and Exam 1 yielded a cyclomatic complexity score of 12, indicating similar levels of logical branching and control flow. In contrast, the Exam 2 task, which required the exclusive use of an LLM, produced a markedly higher complexity score of 23, indicating substantially greater structural complexity. To examine whether this increase in complexity affected student performance, we analyzed score differences between the assignments. Given that the Initial Phase and Exam 1 exhibited identical complexity levels, we focused our statistical comparison on Exam 1 vs. Exam 2, the only pair with a meaningful task complexity variance. Before conducting statistical analysis, we applied the Shapiro–Wilk test to assess the normality of the score distributions. The results indicated significant deviations from

normality in both groups (Exam 1: $W = 0.846$, $p = 0.0001$; Exam 2: $W = 0.892$, $p = 0.0018$). Consequently, we employed the Mann–Whitney U test, a non-parametric alternative to the t -test. The analysis yielded $U = 783.0$, $p = 0.286$, indicating no statistically significant difference in student performance between the two exams, despite the increased task complexity in Exam 2. This finding suggests that LLM assistance may have enabled students to handle more complex programming challenges without a corresponding decline in performance, highlighting the potential of LLMs to mitigate the cognitive load associated with advanced tasks.

4. Results

4.1. Data Overview

As part of this study, we analyzed 37 students' user interactions with LLMs to understand how they seek assistance in solving programming-related problems. The data extraction process involved collecting prompts directed towards the model, with each query representing a request for advice on a specific coding or programming issue. These prompts were then systematically categorized based on the nature of the request to facilitate a deeper understanding of user behavior.

The users' question types were categorized based on the intent behind each prompt, which was inferred through manual review and tagging. To categorize the queries, a manual coding process was used in which two independent reviewers assigned tags to each prompt based on its semantic content and intended purpose. Discrepancies were discussed and resolved through consensus. To enhance consistency and reduce subjective bias, a predefined coding scheme with clear definitions for each category (e.g., Error Fixing, Code Understanding) was developed and used throughout the tagging process.

The following types of prompts were identified, corresponding to distinct areas of programming assistance:

- **Error Fixing (EF):** Queries that sought help identifying and resolving specific bugs or errors in the code. These requests often included error messages or descriptions of unexpected behaviors in the code.
- **Feature Implementation (FI):** Requests for adding new features or functionalities within a codebase. These queries focused on implementing specific functionalities that the users wanted to incorporate into their projects.
- **Code Optimization (CO):** Queries aimed at improving the code's performance, efficiency, or quality. Users sought advice on refactoring or optimizing existing code to meet higher standards of performance or readability.
- **Code Understanding (CU):** Requests for clarification or explanation regarding how a particular piece of code, library, or framework works. These queries indicated that users sought to comprehend complex code segments or third-party tools.
- **Best Practices (BP):** Questions about general coding standards, guidelines, or best practices. These queries focused on improving code quality through adherence to widely accepted programming conventions.
- **Documentation (DOC):** Requests for official documentation, code examples, or references to clarify certain aspects of the code or its dependencies. These queries typically involved users seeking authoritative resources to guide their programming decisions.
- **Concept Clarification (CC):** Queries about broader programming concepts or theoretical principles. These questions were not specific to a particular coding task but focused on understanding fundamental programming ideas.

As noted earlier, prompt classification was conducted independently by two evaluators to reduce the risk of bias. Inter-rater agreement was assessed using Cohen's kappa coefficient [19], which quantifies agreement for categorical data while accounting for chance. As

each prompt could be assigned multiple tags, label-wise Cohen's kappa was calculated for each category. The results indicated excellent agreement beyond chance for EF ($k = 0.786$) and CC ($k = 0.778$), and good agreement beyond chance for DOC ($k = 0.722$) [20]. Additionally, good agreement beyond chance was observed for CU ($k = 0.621$) and CO ($k = 0.540$) [20]. These values demonstrate consistent and reliable classification across evaluators.

To further contextualize our tagging scheme within the prompt engineering literature, we examined how each of the seven query categories identified in this study aligns with established prompt modes used in LLM research [8–10]. For example, queries labelled EF and FI often resembled instruction-based or few-shot prompts, as students frequently described the current state of their code and asked the model to make specific changes. CO queries reflected a mix of instructional and chain-of-thought prompting, especially when students asked for step-by-step refactoring suggestions or performance improvements. Code Understanding and Concept Clarification queries were typically structured as zero-shot requests, where students asked for explanations without providing additional context or examples. Best Practices and Documentation queries also aligned with zero-shot or instruction-based modes, as students sought general standards, guidelines, or references without referencing specific code. By mapping these organically occurring student interactions to known prompting strategies, we bridge the gap between controlled prompt design research and real-world LLM usage in educational programming contexts.

After the data extraction process and the category identification, descriptive statistics of the measured variables used in the analysis were performed. The data were grouped into two conditions: queries where no tag was present and queries where a tag was present. Table 1 shows an overview of the students' prompts with the mean, standard deviation, and sample size.

Table 1. Descriptive statistics of measurement.

Tag	No Tag Present			Tag Present		
	Mean	SD	N	Mean	SD	N
FI	1.970	4.815	5	0.650	3.819	32
BP	0.755	4.100	33	1.438	2.075	4
DOC	0.747	3.941	36	3.750	/	1
CC	1.025	4.567	10	0.756	3.743	27
EF	3.625	5.044	6	0.287	3.508	31
CU	4.688	4.033	4	0.361	3.695	33
CO	1.293	4.429	21	0.219	3.158	16

Table 1 presents the descriptive statistics of the students' queries, highlighting differences in mean values across tagged and untagged queries. While EF and CU were the most frequent queries when no tags were present, their frequency significantly decreased when tagging was introduced. Conversely, DOC queries saw a notable increase in the tagged condition despite having a lower mean when untagged. To further illustrate these patterns, Figure 3 provides a comparative bar chart, visually emphasizing how tagging influenced the frequency of each query type. The chart reinforces that students in the cases where a tag was present tended to ask fewer questions in categories like EF and CU, suggesting that precise tagging may lead to more effective or self-sufficient problem-solving. On the other hand, the increase in DOC queries when tagged indicates that students were more deliberate in seeking structured documentation.

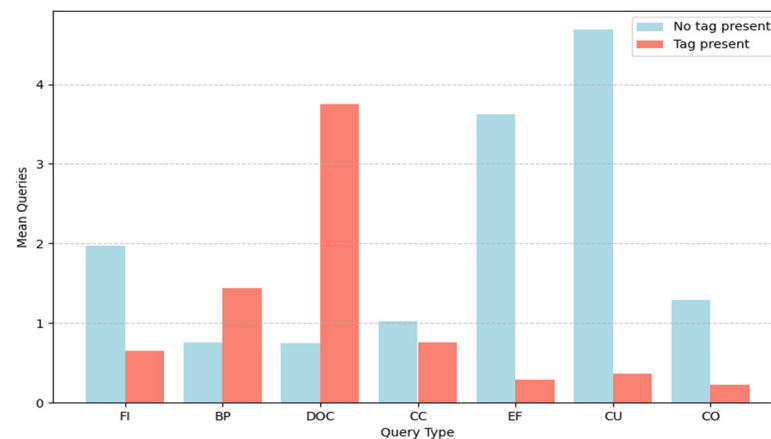


Figure 3. Mean number of queries per category: Tagged vs. untagged conditions. The chart shows how tagging affects query frequency across categories. A notable decrease in Error Fixing (EF) and Code Understanding (CU) queries when tags are present suggests that students formulated more precise prompts, potentially improving the effectiveness of LLM responses.

4.2. Methodological Framework of the Statistical Analysis

As part of the experiment, we first obtained the LLM chat history from the students and extracted their queries for further analysis. Each query was examined to identify and categorize relevant tags, which were then used to assess various aspects of performance. To evaluate the impact of these tags, we conducted several statistical analyses. These included examining the relationship between specific types of queries and the code's functionality and analyzing the correlation between query types and final grades. To ensure a fair comparison, we normalized the performance results using grades from prior assignments. This normalization allowed us to evaluate whether patterns in LLM query usage were independent of the students' existing coding abilities.

The analysis was carried out using R 4.3.3 (R Foundation for Statistical Computing, Vienna, Austria) in RStudio. We employed the Mann–Whitney U test to compare performance outcomes between cases with tagged queries and those without. This non-parametric test was selected due to the non-normal distribution of the data, as indicated by the Shapiro–Wilk test ($p < 0.05$). The Mann–Whitney U test provided a robust method for assessing the impact of query tagging on performance, accommodating the non-normality of the data and offering insights into how the presence of specific tags influenced student outcomes.

4.3. Results and Interpretations

First, the histograms in Figure 4 illustrate the distribution of grade differences across all query categories included in the analysis. Each category is represented by a pair of histograms: blue bars indicate the grade differences for prompts that were not assigned to the given category, while red bars show the grade differences for prompts that were assigned to that category. These visualizations provide an initial impression of the impact that different query types may have had on the grading outcomes. In some cases (especially EF and CU), noticeable differences between the red and blue distributions suggest potential effects of category membership. However, we conducted formal statistical analyses to move beyond visual interpretation and rigorously assess whether these observed differences are statistically meaningful or simply the result of random variation. This approach allows us to determine whether the patterns seen in the histograms reflect genuine relationships in the data.

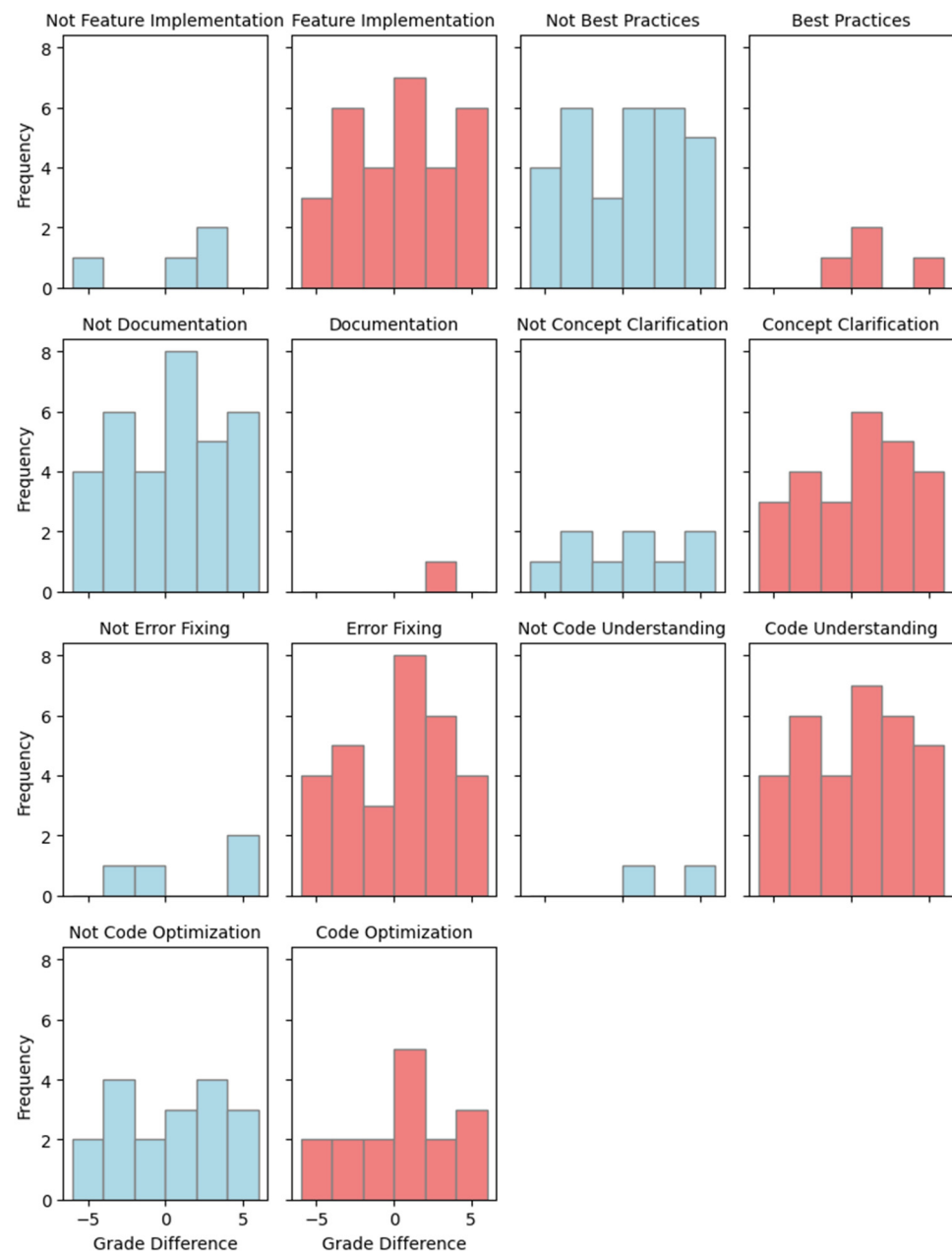


Figure 4. Distribution of grade differences by query category. Each histogram pair contrasts grade outcomes between students whose queries fell into a given category versus those whose queries did not. Red bars indicate tagged queries, while blue bars indicate untagged ones. Notable shifts for Error Fixing (EF) and Code Understanding (CU) suggest a measurable performance gain linked to precise, well-categorized prompts.

This study employed the Mann–Whitney U test to evaluate the differences in student performance based on the types of queries made to the LLM. This non-parametric test was selected due to the non-normal distribution of the data, as indicated by the Shapiro–Wilk test ($p < 0.05$). The test provided a robust method for assessing whether specific query categories significantly influenced students’ coding performance.

The analysis focused on the relationship between query types and two primary outcomes: the production of runnable code and overall assignment performance. The following query categories were analysed: Feature Implementation (FI), Best Practices (BP), Documentation (DOC), Concept Clarification (CC), Error Fixing (EF), Code Understanding

(CU), and Code Optimization (CO). Results were divided into tagged and untagged queries to investigate the impact of categorization on performance.

The statistical results for each query type are summarized in Table 2 below. These results highlight the U-values and *p*-values obtained from the Mann–Whitney U test, indicating the significance of differences in performance outcomes between tagged and untagged queries.

Table 2. Average ranks and Mann–Whitney U test for each query type.

Tag	Avg Ranks		Mann–Whitney		
	No Tag	Tag	U	Z	<i>p</i>
FI	14.4	19.7	90.50	−0.98	0.328
BP	18.3	24.5	56.50	−0.40	0.688
DOC	18.7	30.0	9.00	−0.24	0.813
CC	19.2	18.9	140.00	−0.77	0.439
EF	28.5	17.2	135.50	−2.03	0.042
CU	30.3	17.6	105.00	−2.17	0.030
CO	22.5	14.4	189.50	−1.13	0.260

The results show that two categories, EF and CU, exhibited statistically significant differences. For EF, the Mann–Whitney U statistic was 135.50, with a *p*-value of 0.042, indicating that tagged queries in this category significantly improved students’ ability to produce runnable code. Similarly, for CU, the Mann–Whitney U statistic was 105.00, with a *p*-value of 0.030, suggesting that tagged queries focusing on understanding code concepts led to better performance outcomes. These findings emphasize the importance of precise and deliberate tagging [21] in improving the effectiveness of LLM interactions. This pattern aligns with recent findings in prompt engineering research. As shown in the work by [22], high-specificity prompts, those that include clear instructions, constraints, and objectives, tend to generate more accurate and relevant LLM outputs, whereas low-specificity prompts introduce ambiguity that can lead to irrelevant or erroneous completions. This may help explain the relatively higher variability and lower effectiveness seen with CU-tagged tasks, which may sometimes lack sufficient clarity or constraint. This interpretation aligns with educational research on prompt scaffolding. Wang et al. [21] demonstrate that multi-step, conversational prompting improves contextual understanding and reduces cognitive load in LLM interactions, supporting logical reasoning and user engagement. These characteristics are particularly helpful in tasks like EF, where iterative refinement and stepwise logic are essential. Therefore, the superior performance observed in EF-tagged queries may be attributed to the benefits of such scaffolding structures, which help both the user and the model focus on discrete sub-tasks in complex problem-solving contexts. Finally, the study by Tian and Zhang [23] provides additional insight into LLM behaviour. They note that models may drift from their initial instruction over longer outputs unless the prompt is explicitly anchored. This phenomenon of “attention dilution” supports the interpretation that well-anchored tasks, such as EF, are more robust to drift and yield better outcomes, whereas ambiguous prompts like those in CU queries are more vulnerable to deviation from intended goals. Together, these theoretical insights suggest that the observed EF/CU differences stem from a combination of prompt design (specificity and scaffolding) and intrinsic LLM behaviour. These findings carry practical implications: they highlight the importance of precise prompt engineering and suggest that educators and developers should prioritize anchored and scaffolded prompts for tasks requiring code comprehension or error resolution.

For the other query types, no statistically significant differences were observed. While FI and CO showed trends toward better outcomes with tagging, their p -values were above the 0.05 threshold, indicating insufficient evidence to conclude a significant impact. The DOC category had a particularly small sample size, limiting the interpretability of its results. This outcome aligns with findings in recent prompt engineering research, which suggest that not all coding tasks benefit equally from LLM assistance. Murr et al. [22] note that tasks requiring open-ended generation, such as feature implementation, often depend heavily on the user's ability to frame precise constraints and goals. Without detailed specification, LLMs may default to generic or suboptimal solutions, reducing the marginal benefit of tagging. Similarly, Tian and Zhang [23] show that prompt anchoring is most effective when the task involves error resolution or explanation, while its utility diminishes in tasks that require creative or strategic planning, as is common in optimization and feature design. Finally, for DOC, the limited sample size constrains the statistical power of the test, but this task type may also inherently require less LLM assistance, as it often depends more on context familiarity than reasoning or generation complexity.

To assess the impact of ChatGPT on developers' coding performance, we compared the difference between grades received on the two separate exams in the controlled and experimental phase: one completed without the use of ChatGPT (an easier exam) and one with ChatGPT's assistance (a more challenging exam). By using two exams of differing difficulty, we aim to control for baseline knowledge and isolate the effect of ChatGPT usage. The difference in grades was calculated as the score from the exam without ChatGPT subtracted from the score on the exam with ChatGPT. This method allows us to focus on how ChatGPT may bridge the performance gap when tackling more complex coding problems, regardless of the developer's inherent knowledge or skills.

In this analysis, we examined how different types of queries (tags) related to React.js impact developers' performance on a controlled exam with and without the help of ChatGPT. The Mann–Whitney U test was used to compare the differences in performance between cases where no tag was present and cases where a tag was specified. By examining Tables 1 and 2, we can note that among the tags analyzed, significant differences were found for EF and CU. For EF, developers showed a larger grade difference without ChatGPT compared to with ChatGPT (mean = 3.625, SD = 5.044, N = 6) compared to when a tag was present (mean = 0.287, SD = 3.508, N = 31), with a statistically significant result ($U = 135.50$, $p = 0.042$). A similar trend was seen for CU, where developers without ChatGPT performed substantially better (mean = 4.688, SD = 4.033, N = 4) than those who used ChatGPT with the CU tag (mean = 0.361, SD = 3.695, N = 33), with a significant difference ($U = 105.00$, $p = 0.030$). For other tags, such as FI, BP, DOC, CC, and CO, the differences in performance were not statistically significant ($p > 0.05$), indicating that ChatGPT's contribution may not be as impactful in these areas, or the sample sizes were insufficient to draw firm conclusions.

These findings suggest that ChatGPT could be particularly helpful for tasks involving code understanding and error fixing, as developers who used ChatGPT in these contexts performed closer to their scores on a more difficult exam, indicating that the assistance potentially mitigated the difficulty gap. Organizations or individual developers could leverage this insight by promoting the use of AI assistance for error resolution and comprehension tasks while focusing on manual learning and problem-solving skills for feature implementation, concept clarification, and other areas where the AI's assistance shows less significant impact. This could lead to more efficient workflows and a tailored use of AI for specific coding tasks. This division of labour aligns with the prompt specificity framework described by Murr et al. [22], which argues that the more constrained and clearly defined a problem is, the more reliable and relevant the model output becomes. In contrast, when prompts are vague, such as in open-ended feature development, the model

may generate plausible but ungrounded suggestions, which could confuse rather than assist novice developers.

Students who used tagged queries for EF were more likely to submit runnable code compared to those who used untagged queries. This suggests that tagging allows students to specify their problems more precisely, enabling the LLM to generate more targeted and helpful responses. This result highlights the critical role of query clarity in debugging tasks, particularly for novice programmers. From a theoretical perspective, this aligns with the principles of selective attention and anchoring explored by Tian and Zhang [23], who find that prompts that include specific goals and context act as anchors that constrain the model's focus throughout the generation process. This focused output generation increases task success rates, especially in goal-directed tasks like debugging.

Similarly, tagged queries for CU were associated with better overall performance. This finding indicates that students who actively sought to deepen their comprehension of code concepts benefited significantly from LLM assistance. The educational implications of this result are notable, as it underscores the potential of LLMs to enhance conceptual understanding alongside practical coding skills. In particular, this supports the use of AI as a metacognitive aid: by facilitating reflection, explanation, and iterative reasoning, LLMs may function not only as coding tools but also as scaffolding for conceptual development. As Wang et al. [21] argue, when LLMs are used in educational contexts with well-structured prompts, they can foster higher-order thinking by breaking down abstract ideas into concrete reasoning steps, making them powerful allies in teaching foundational computer science concepts.

4.4. Examples of Tagged Queries

This section provides examples of tagged queries for each category to illustrate further the types of queries used. These examples showcase the specificity and focus of well-tagged queries, which likely contributed to the improved performance observed in specific categories. Note: For clarity, all prompts are translated from the Slovenian language.

Feature Implementation (FI) example prompt: "Implement an application for managing task lists with support for user accounts. Use React, React Router, and TypeScript. Requirements include allowing users to log into the system using the API: <https://reqres.in/> (accessed on 14 June 2025). Once logged in, the username should be saved in localStorage."

This prompt directly asks for developing a task management application using specific technologies like React, React Router, and TypeScript. It entails creating features such as user authentication, task list display, and interactions with an external API. These elements align with the core tasks in feature implementation, as the user must build an application from a conceptual requirement into a functional software solution. In this context, the need to integrate user login and task management implies a significant level of application design and coding.

Error Fixing (EF) example prompt: "react-router-dom" has no exported member 'Switch'.ts(2305) Module 'react-router-dom' has no exported member 'useHistory'. Fix these import errors."

The presented prompt showcases a clear case of debugging and error resolution, focusing on fixing TypeScript import errors with the React Router library. It reflects the user's struggle with module imports, a common issue developers face when library updates affect existing codebases. Solving such errors requires an understanding of the library's current API and how to adapt the existing code accordingly. In this scenario, the emphasis is on identifying the discrepancies between expected and available exports in react-router-dom, marking it as an EF task.

Code Understanding (CU) example prompt: “How do I set it up so that when a user opens the application, they are immediately redirected to /login and only then able to access other routes?”

This prompt highlights the user’s challenge in understanding how to implement routing logic within React that aligns with the user’s authentication status. It illustrates a need to grasp how React Router can be utilized to conditionally redirect users based on their login state, navigating between different parts of the application accordingly. The pursuit of this knowledge reflects the user’s requirement to internalize effective routing strategies that are central to building user-friendly navigational flows within web applications, deeming it a CU task.

Code Optimization (CO) example prompt: “Use React routing for navigation between different parts of the application, including login, main page, and task adding page.”

Although this prompt primarily addresses implementation, it inherently suggests optimizing and structuring the application’s routing system. The prompt encourages the developer to consider efficient routing practices and component organization by emphasizing the use of React’s routing capabilities for seamless navigation. Hence, while not overtly focused on optimization, there is an implicit call to improve the routing architecture, making the application more scalable and maintainable.

Concept Clarification (CC) example prompt: “What is Axios, and can I achieve this without using Axios?”

This question arises when the developer is uncertain about the necessity of a specific library (Axios) for performing HTTP requests in their application.

5. Discussion

The results indicate that LLMs play a dual role in assisting students with web development tasks. On the one hand, students who focused on error fixing were more likely to produce functioning applications, suggesting that LLMs excel at helping students resolve bugs and issues efficiently. On the other hand, the improvement in performance for students who asked about code understanding points to the educational value of AI, as these students could leverage the tool to deepen their knowledge of the underlying concepts. These results suggest that LLMs are particularly effective debugging and conceptual clarification tools, reinforcing their value in software engineering education. More specifically, our research shows the following.

RQ1: How does using LLM-based coding assistance impact the likelihood of students producing runnable code, considering their prior coding experience?

Using tagged queries for EF and CU significantly increased the rate of runnable submissions, indicating that LLM-based assistance can improve coding outcomes when students frame their requests around debugging or conceptual comprehension. This partially supports H1: LLM-based assistance does boost runnable code production overall, but its effects are strongest in contexts where the problem is clearly specified.

RQ2: How does the formulation of queries when using LLM-based coding assistance influence the effectiveness of the assistance?

The statistically significant improvements for EF ($U = 135.50$, $p = 0.042$) and CU ($U = 105.00$, $p = 0.030$) confirm H2: precise query formulation substantially enhances the efficacy of LLM support. In contrast, other query types showed no significant differences, highlighting the critical role of clarity and specificity in prompt design for debugging and understanding tasks.

RQ3: How does prior coding experience affect the benefits students receive from LLM-based coding assistance?

While novice students exhibited the most pronounced gains in EF and CU conditions, differences across experience levels in other categories did not reach statistical significance. Thus, H3 is only partially supported: less experienced students benefit markedly from well-tagged queries in debugging and comprehension, but further research with larger samples is needed to clarify how expertise influences LLM-driven improvements in feature implementation, optimization, or best-practice inquiries.

Interestingly, the findings show that query types like feature implementation or code optimization did not significantly correlate with better performance. This might imply that students are not yet fully equipped to translate AI-generated feature suggestions into well-functioning code or that optimizing code might not have been a priority in the limited time frame of the experiment. Additionally, optimization often requires a broader understanding of trade-offs, which may not be fully conveyed through LLM-generated responses.

Furthermore, the results suggest that query formulation skills are critical for maximizing the benefits of AI-assisted programming. Students who specified their needs, especially in error fixing and conceptual understanding, received more actionable guidance from the LLM. This finding reinforces the importance of prompt engineering as a skill in modern software development and education. Additionally, the study highlights the complementary role of LLMs in education. While LLMs can assist with debugging and explaining concepts, they are not a substitute for fundamental programming skills. Educators should encourage students to combine AI assistance with traditional problem-solving approaches to prevent over-reliance on automated tools.

The findings of our study complement existing research [24] by providing a domain-specific examination of AI-assisted tasks in software development, whereas prior studies focus on AI's role in supporting creative processes within early-stage startups. The existing research emphasizes the importance of applying structured prompt patterns, such as Flipped Interaction and Cognitive Verifier, to stimulate divergent thinking and enhance brainstorming activities. In contrast, our study explores how AI, specifically ChatGPT, can alleviate challenges in coding tasks, particularly error resolution and code comprehension. Both types of studies highlight the value of prompt engineering in maximizing the effectiveness of LLMs; however, our results indicate that AI's impact is more substantial in technical tasks where developers can benefit from automated assistance in identifying and resolving errors. This differs from the creative context, where AI's role is to provoke exploratory thinking rather than provide conclusive answers. Taken together, the findings from both types of studies emphasize that the success of prompt engineering is not purely technical but is also influenced by socio-cultural and human factors, with different domains necessitating tailored approaches to optimize AI's potential.

5.1. Theoretical and Practical Implications

This study offers meaningful theoretical contributions to computer science education, prompt engineering, and human–AI interaction. It empirically demonstrates that not all interactions with LLMs are equally effective, rather, the type and clarity of the query significantly influence learning outcomes and task success. Specifically, the study found that students who formulated queries related to EF and CU showed statistically significant improvements in both the functionality and correctness of their code. These findings support and extend existing theories in cognitive learning science by suggesting that AI-assisted learning is most effective when it targets immediate problem resolution and conceptual clarity. Moreover, the research highlights that prompt formulation is not merely a technical interaction but a cognitively demanding task that affects how learners extract meaningful feedback from LLMs. This aligns with emerging frameworks in digital pedagogy that treat prompt engineering as a critical thinking skill in itself, reinforcing

the notion that generative AI does not eliminate the need for metacognitive effort but redistributes it toward interaction design. Thus, this work lays a foundation for future research into how different prompt categories map onto various levels of programming cognition (e.g., comprehension vs. synthesis).

The practical implications of this study are particularly relevant for educators, curriculum designers, and developers of AI-based coding tools. First, the findings suggest that teaching students how to ask better questions, especially those related to debugging and code comprehension, can significantly improve their ability to produce runnable and correct code, even when prior programming experience is limited. This is critical in introductory programming courses where students frequently struggle with diagnosing problems and understanding complex code behaviour. Embedding targeted exercises in prompt formulation or using LLMs for conceptual clarification could be a low-cost, high-impact intervention to support student learning. Additionally, the study demonstrates that tagging or explicitly categorizing one's query type (e.g., EF, CU) can lead to more deliberate and structured interactions with LLMs. Tool designers could leverage this insight by integrating query-type selection interfaces or guided prompt templates into AI coding assistants to help users refine their input more effectively. From a broader organizational perspective, development teams that adopt LLMs into their workflows might consider offering internal training on prompt strategy or implementing lightweight tagging systems to improve the efficiency and reliability of AI-generated output. Ultimately, this research reinforces the idea that human factors, especially intentional communication, are central to unlocking the value of AI assistance in software development and education.

5.2. Limitations and Future Work

This study is subject to several limitations that should be taken into consideration when interpreting the findings.

First, the participant sample comprised second-year undergraduate students with varying degrees of prior programming experience. Although this population is appropriate for evaluating the educational utility of LLMs, the generalizability of the results to more experienced developers or professional settings remains limited. Future studies should consider a broader demographic, including advanced students and industry practitioners, to assess whether similar effects are observed across different experience levels.

Second, the programming tasks used in the study were intentionally simplified to fit the academic context and time constraints. These assignments were designed to reinforce specific technical concepts rather than replicate the complexity of real-world software development. Consequently, they did not incorporate aspects such as ambiguous or evolving requirements, team-based collaboration, scalability considerations, or deployment workflows. Follow-up research should employ more realistic and open-ended programming scenarios to examine the robustness of LLM assistance in production-like environments.

Third, the investigation focused exclusively on ChatGPT as the LLM used by participants. While ChatGPT is one of the most widely adopted models, its behaviour and performance may not be representative of other generative systems. Alternative LLMs, such as Claude, Gemini, Copilot, or open-source models, differ in terms of architecture, training data, and interaction style. Future research should include comparative analyses across multiple LLM platforms to evaluate whether the observed effects related to query type hold consistently.

Finally, the analysis was limited to the categorization of query types and their relationship to performance outcomes. The study did not include a systematic evaluation of the correctness, completeness, or pedagogical quality of the LLM-generated responses. Future work should examine the content of these responses using expert evaluation criteria

to better understand how different types of queries influence the educational value and functional reliability of LLM outputs.

Collectively, these limitations suggest several directions for future research, including extending participant diversity, increasing task realism, comparing multiple LLMs, formalizing annotation reliability, and evaluating AI response quality. Addressing these areas would contribute to a more comprehensive understanding of how conversational LLMs can best support programming education and software development workflows.

6. Conclusions

This study highlights the significant impact of query types on students' success when using Large Language Models as coding assistants.

The study revealed that Error Fixing queries dramatically increased the probability of producing both syntactically correct and functionally accurate code. Queries aimed at Code Comprehension also yielded significant performance gains, even after adjusting for students' prior coding experience. In contrast, other query categories, such as Feature Implementation, Best Practices, Documentation, Concept Clarification, and Performance Optimization, exhibited more variable, context-dependent benefits.

To leverage these insights, computer and information science educators should introduce explicit query design instructions. Embedding AI-assisted debugging and conceptual explanation exercises into programming assignments will highlight LLMs' most impactful use cases.

Similarly, software development teams can benefit from prompt-engineering workshops emphasizing precise, targeted questions for troubleshooting and code analysis. Adopting the systematic tagging and classification of queries can further streamline workflow and improve the relevance of AI-generated solutions.

Our findings confirm and extend the existing literature on AI-supported education by evidencing how specific types of user interactions can significantly influence learning effectiveness. It reinforces theories highlighting the need for human oversight and efficient query engineering in utilizing generative AI tools.

This study was limited to second-year undergraduate informatics students, which may constrain the generalizability of our conclusions. Future work should broaden the participant pool and examine how query-formulation proficiency evolves with sustained LLM use.

In summary, LLMs can substantially accelerate coding tasks, especially error resolution and conceptual clarification, but realizing their full potential relies on deliberate query formulation and structured curricular integration.

Author Contributions: Conceptualization, V.T. and G.J.; Data curation, S.K. and G.J.; Formal analysis, S.K.; Investigation, V.T. and P.R.; Methodology, S.K. and G.J.; Resources, V.T. and G.J.; Validation, V.T. and S.K.; Visualization, S.K.; Writing—original draft, V.T., S.K., P.R. and G.J.; Writing—review & editing, V.T., S.K., P.R. and G.J. All authors have read and agreed to the published version of the manuscript.

Funding: The authors acknowledge the financial support from the Slovenian Research and Innovation Agency (ARIS) (Research Core Funding No. P2-0057).

Institutional Review Board Statement: The study was conducted in accordance with the Rules of Procedure of the Ethics Committee of the Institute of Informatics, Faculty of Electrical Engineering, Computer Science at the University of Maribor, and approved by the Research Ethics Committee at the Institute of Informatics, Faculty of Electrical Engineering and Computer Science, University of Maribor (protocol code 3/163-GŠ/2025 0304-ENG, date of approval 3 March 2025).

Informed Consent Statement: Informed consent was obtained from all subjects involved in the study. During the preparation of this work, the author(s) used ChatGPT to improve the manuscript's readability and language. After using this tool/service, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the published article.

Data Availability Statement: The raw data supporting the conclusions of this article will be made available by the authors on request.

Acknowledgments: The authors acknowledge the use of Generative AI and AI-assisted technologies in the writing process. During the preparation of this work, the authors used ChatGPT to improve the manuscript's readability and language. After using this tool/service, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

Conflicts of Interest: The authors declare no conflicts of interest.

Appendix A. Instructions for Assignments That Were Used in the Controlled Phase

The assignments in the controlled phase were comprehensive practical exercises designed to consolidate the skills developed in earlier tasks from the initial phase of the study. The instructions below were used to describe the assignments in the controlled phase:

Exam 1

Implement an application for viewing and adding tasks to a list. A task consists of a name and a description. Use React and TypeScript.

Requirements:

- The application should include a main page that displays the task list. Each item in the list should include the task name and description.
- The application should also have a separate page for adding new tasks to the list. The form should include fields for the name and description. When the user adds a new task, it should also appear on the main page.
- Use props and state lifting to pass data between components.
- You do not need to use the React Router library for navigating between pages.

Tips:

- Break the application into smaller components and use composition to build the user interface.
- Test your components as you build them to ensure they work as expected. Plan the application's architecture before you start coding to avoid getting stuck later.
- The structure of the interfaces is up to you.

Exam 2

Implement a task management application with user account support. Use React, React Router, and TypeScript.

Requirements:

- The application should allow users to log in. To verify existing users, use the following API: <https://reqres.in/>. Once the user successfully logs in, store their username in localStorage. (route: "/login").
- After logging in, the user should have access to the main page, which displays the task list. Each task should include a name and description. (route: "/", redirect if the user is not logged in).
- The application should also have a separate page for adding new tasks to the list. The form should include fields for the task name and description. When the user adds a new task, it should also appear on the main page. (route: "/add").
- Use React routing for navigation between different parts of the application, including the login page, the main page, and the page for adding new tasks.

- Use the Context API to manage the login state within the application.
- Tips:
- Break the application into smaller components and use composition to build the user interface.
 - Test your components as you build them to ensure they work as expected.
 - Plan the application's architecture before you start coding to avoid getting stuck later.
 - Use props and state lifting to pass data between components.
 - The structure of the interfaces is up to you.

Appendix B. Subcriteria for Grading Correctness

The correctness of each solution was evaluated based on the following subcriteria, all of which reflect fundamental React concepts and best practices:

- Components: Whether the solution effectively used component-based approach, ensuring the code is modular, reusable, and adheres to React's component-based architecture.
- Lifting state: Whether the solution properly implemented state management, including lifting state up to the nearest common ancestor, when necessary, to ensure data flows smoothly between components.
- Functional Correctness: Whether the solution meets the specified requirements and functions as expected without significant bugs or errors.
- Typescript: Whether the solution effectively used TypeScript to ensure type safety and improve code reliability, including proper use of interfaces, types, and type annotations.
- Functionalities: Whether the implemented features and functionalities met the assignment requirements, and if additional functionalities were handled well within the React ecosystem.
- React Router: Whether the solution implemented React Router correctly, ensuring smooth navigation between different views or pages within the app.
- Code Quality: Whether the code was clean, well-organized, and followed best practices, including proper naming conventions, indentation, and adherence to DRY (Don't Repeat Yourself) principles.

References

1. Wang, W.; Ning, H.; Zhang, G.; Liu, L.; Wang, Y. Rocks Coding, Not Development: A Human-Centric, Experimental Evaluation of LLM-Supported SE Tasks. *Proc. ACM Softw. Eng.* **2024**, *1*, 699–721. [\[CrossRef\]](#)
2. Rasnayaka, S.; Wang, G.; Shariffdeen, R.; Iyer, G.N. An Empirical Study on Usage and Perceptions of LLMs in a Software Engineering Project. *arXiv* **2024**, arXiv:2401.16186. <http://arxiv.org/abs/2401.16186>.
3. Peláez-Sánchez, I.C.; Velarde-Camaqui, D.; Glasserman-Morales, L.D. The impact of large language models on higher education: Exploring the connection between AI and Education 4.0. *Front. Educ.* **2024**, *9*. [\[CrossRef\]](#)
4. Champa, A.I.; Rabbi, M.F.; Nachuma, C.; Zibran, M.F. ChatGPT in Action: Analyzing Its Use in Software Development. In Proceedings of the 21st International Conference on Mining Software Repositories, Lisbon, Portugal, 15–16 April 2024; Association for Computing Machinery: New York, NY, USA, 2024; pp. 182–186. [\[CrossRef\]](#)
5. Rajbhoj, A.; Somase, A.; Kulkarni, P.; Kulkarni, V. Accelerating Software Development Using Generative AI: ChatGPT Case Study. In Proceedings of the 17th Innovations in Software Engineering Conference, Bangalore, India, 22–24 February 2024; Association for Computing Machinery: New York, NY, USA, 2024. [\[CrossRef\]](#)
6. White, J.; Hays, S.; Fu, Q.; Spencer-Smith, J.; Schmidt, D.C. ChatGPT Prompt Patterns for Improving Code Quality, Refactoring, Requirements Elicitation, and Software Design. In *Generative AI for Effective Software Development*; Nguyen-Duc, A., Abrahamsson, P., Khomh, F., Eds.; Springer Nature: Cham, Switzerland, 2024; pp. 71–108. [\[CrossRef\]](#)
7. Della Porta, A.; Lambiase, S.; Palomba, F. Do Prompt Patterns Affect Code Quality? A First Empirical Assessment of ChatGPT-Generated Code. *arXiv* **2025**, arXiv:2504.13656. <https://arxiv.org/abs/2504.13656>.
8. Brown, T.B.; Mann, B.; Ryder, N.; Subbiah, M.; Kaplan, J.; Dhariwal, P.; Neelakantan, A.; Shyam, P.; Sastry, G.; Askell, A.; et al. Language Models are Few-Shot Learners. *arXiv* **2020**, arXiv:2005.14165. <https://arxiv.org/abs/2005.14165>.

9. Liu, P.; Yuan, W.; Fu, J.; Jiang, Z.; Hayashi, H.; Neubig, G. Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing. *arXiv* **2021**, arXiv:2107.13586. <https://arxiv.org/abs/2107.13586>.
10. Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Ichter, B.; Xia, F.; Chi, E.; Le, Q.; Zhou, D. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *arXiv* **2023**, arXiv:2201.11903. <https://arxiv.org/abs/2201.11903>.
11. White, J.; Fu, Q.; Hays, S.; Sandborn, M.; Olea, C.; Gilbert, H.; Elnashar, A.; Spencer-Smith, J.; Schmidt, D.C. A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT. *arXiv* **2023**, arXiv:2302.11382. <https://arxiv.org/abs/2302.11382>.
12. Henrickson, L.; Meroño-Peñuela, A. Prompting meaning: A hermeneutic approach to optimising prompt engineering with ChatGPT. *AI Soc.* **2025**, *40*, 903–918. [[CrossRef](#)]
13. Liu, J.; Wang, K.; Chen, Y.; Peng, X.; Chen, Z.; Zhang, L.; Lou, Y. Large Language Model-Based Agents for Software Engineering: A Survey. *arXiv* **2024**, arXiv:2409.02977. <https://arxiv.org/abs/2409.02977>.
14. Biggs, J.B.; Tang, C. *Teaching for Quality Learning at University*, 4th ed.; Open University Press: London, UK, 2011.
15. Zamfirescu-Pereira, J.D.; Wong, R.Y.; Hartmann, B.; Yang, Q. Why Johnny Can't Prompt: How Non-AI Experts Try (and Fail) to Design LLM Prompts. In Proceedings of the Conference on Human Factors in Computing Systems, Hamburg, Germany, 23–28 April 2023. [[CrossRef](#)]
16. Sawalha, G.; Imran, T. Shoufan Analyzing student prompts and their effect on ChatGPT's performance. *Cogent Educ.* **2024**, *11*, 2397200. [[CrossRef](#)]
17. Theophilou, E.; Koyuturk, C.; Yavari, M.; Bursic, S.; Donabauer, G.; Telari, A.; Testa, A.; Boiano, R.; Hernandez-Leo, D.; Ruskov, M.; et al. Learning to Prompt in the Classroom to Understand AI Limits: A Pilot Study. In *AIxIA 2023—Advances in Artificial Intelligence*; Basili, R., Lembo, D., Limongelli, C., Orlandini, A., Eds.; Springer Nature: Cham, Switzerland, 2023; pp. 481–496.
18. SonarQube Cloud Online Code Review as a Service Tool | Sonar. Available online: <https://www.sonarsource.com/products/sonarcloud/> (accessed on 9 June 2025).
19. Cohen, J. A Coefficient of Agreement for Nominal Scales. *Educ. Psychol. Meas.* **1960**, *20*, 37–46. [[CrossRef](#)]
20. Fleiss, J.L.; Levin, B.; Paik, M.C. *The Measurement of Interrater Agreement*; John Wiley & Sons, Inc.: Hoboken, NJ, USA, 2004.
21. Wang, T.; Zhou, N.; Chen, Z. Enhancing Computer Programming Education with LLMs: A Study on Effective Prompt Engineering for Python Code Generation. *arXiv* **2024**, arXiv:2407.05437. <http://arxiv.org/abs/2407.05437>.
22. Murr, L.; Grainger, M.; Gao, D. Testing LLMs on Code Generation with Varying Levels of Prompt Specificity. *arXiv* **2023**, arXiv:2311.07599. <https://arxiv.org/abs/2311.07599>.
23. Tian, Y.; Zhang, T. Selective Prompt Anchoring for Code Generation. *arXiv* **2024**, arXiv:2408.09121. <https://arxiv.org/abs/2408.09121>.
24. Wang, X.; Attal, M.I.; Rafiq, U.; Hubner-Benz, S. Turning Large Language Models into AI Assistants for Startups Using Prompt Patterns. In *Agile Processes in Software Engineering and Extreme Programming—Workshops*; Kruchten, P., Gregory, P., Eds.; Springer Nature: Cham, Switzerland, 2024; pp. 192–200.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.